

COMPUTER ORGANIZATION AND ARCHITECTURE LAB

CS2610 (LAB 11 REPORT)

1. BRANCH PREDICTOR IMPLEMENTATION

1.1 IMPLEMENTATION OVERVIEW

This was a continuation of the Lab 10 (two stage RISC-V pipeline structure). Here, we have added a dynamic branch predictor to a two stage RISC-V pipeline structure. A branch predictor has two components– a Branch Target Buffer (BTB) and a Branch History Table (BHT). BTB is generally needed in a 5 stage pipeline structure. But, we can also implement one using caches.

Here, in this lab, we have implemented a Branch History Table (BHT) along with a 2-bit saturating counter to predict branch predictions and also to reduce the control hazard penalties that might occur in the two-stage pipeline structure of RISC-V.

1.2 ARCHITECTURAL OVERVIEW

The implementation contains a 2-bit saturating counter. There are 4 states in this 2-bit saturating counter. Since it's a 2-bit counter, thus, it has $2^2 = 4$ possible branch outcomes and thus 4 possible states are possible.

1.2.1 STATES OF THE SATURATING COUNTER

- (a) **00** - represents BRANCH **STRONGLY NOT TAKEN**
- (b) **01** - represents the state **WEAKLY NOT TAKEN**
- (c) **10** - represents the state **WEAKLY TAKEN**
- (d) **11** - represents BRANCH **STRONGLY TAKEN**

1. **STRONGLY NOT TAKEN**- indicates that the conditional branch instruction did not jump to a new address but instead continued to the next sequential instruction, avoiding the overhead of branching
2. **WEAKLY NOT TAKEN**- Indicates that the conditional branch instruction is predicted to continue to the next sequential instruction, but with low confidence and can change with a single differing outcome.
3. **WEAKLY TAKEN**- Indicates that the conditional branch instruction is predicted to jump to a target address, but with low confidence and can change with a single differing outcome.
4. **STRONGLY TAKEN**- indicates that the conditional branch instruction jumps to a target address instead of continuing sequentially.

1.2.2 PREDICTION LOGIC and ARCHITECTURE DETAILS

LOGIC - If the MSB of the counter = 1, then TAKEN.

ARCHITECTURE DETAILS - (a) Counter Size = 2 bits

(b) Number of Branch History Table entries = 64

1.2.3 ARCHITECTURE DIAGRAM AND IMPLEMENTATION DETAILS

IMPLEMENTATION DETAILS - Since, it's a two-stage RISC-V pipeline structure. The first stage will have fetch and decode, while stage 2 will have execute and writeback.

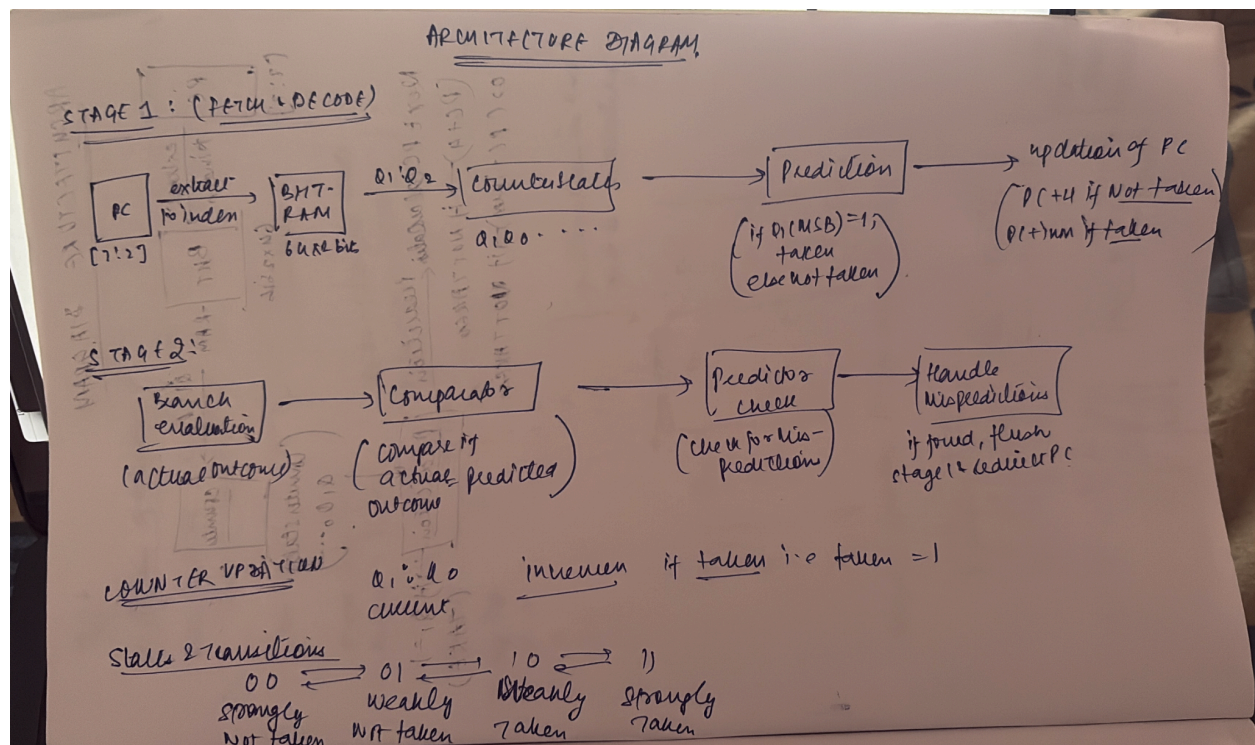
STAGE 1: FETCH AND DECODE

- We extract the 2 to 7 bits to PC to index our branch history table (BHT)
- Then we read the 2-bit counter value from our BHT
- Apply the prediction logic
- Branch is predicted to be TAKEN if the MSB of the counter value is 1
- Else the branch is predicted to be NOT TAKEN
- If it's taken, then we compute the branch target address using PC + B-type immediate
- If it's not taken, then we move to the next instruction sequentially
- Update the PC based on the prediction

STAGE 2: EXECUTE AND WRITEBACK

- We now evaluate the branch condition (zero flag comparison)
- Then, we determine the actual branch outcome
- We finally compare it with the prediction we got from Stage 1
- If a misprediction occurs, then we flush the pipeline
- Redirect the PC after flushing
- We now increment the Branch History Table counter if the prediction was taken
- Decrement is not taken

ARCHITECTURE DIAGRAM-



ARCHITECTURE- (Properties of the implemented BHT)

- Number of entries: 64 => Address Bit Width = 6
- Each entry: 2-bit saturating counter => Data Bit Width = 2
- Indexed using lower bits of PC (e.g., P C[7 : 2])

NOTE: We are calculating the number of mispredictions in Stage 1 only but the flushing of the pipeline (if misprediction occurs) is done in stage 2.

2. RESULTS

In the results, I have reported, the total number of branch instructions (calculated using a counter in stage 1), total number of cycles (calculated using counter in core), total number of mispredictions (calculated using another counter in stage 1 only), total number of correct predictions (*correct predictions = Total number of instructions - number of mispredictions*), accuracy (*accuracy = correct predictions/ total number of branch instructions *100*).

2.1 MATMUL TESTCASE

PARAMETER	VALUE
Total number of branch instructions	34625
Total number of Cycles	35443
Total number of mispredictions	818
Total number of Correct Predictions	2841
Prediction accuracy	77.4%

2.2 MUL_ASSIGN TESTCASE

PARAMETER	VALUE
Total number of branch instructions	155071
Total number of Cycles	155519
Total number of mispredictions	448
Total number of Correct Predictions	5191
Prediction accuracy	92.06%